ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ГОРОДА МОСКВЫ
«Колледж связи № 54» имени П.М. Вострухина

ОТЧЕТ О ВЫПОЛНЕНИИ

по дисциплине

“Внедрение и поддержка компьютерных систем”

Практической работы №7

«Мини-проект «Список задач» (итоговая работа)»

Выполнил: ст.гр. 3-ИСП11-43

ФИО: Юхимчук Евгений Викторович

Специальность: 09.02.07

Информационные системы и программирование

Проверил: преподаватель Кандыбаева

Москва

2026

Практическая работа №7. Мини-проект «Список задач» (итоговая работа)

Ссылка на репозиторий: https://github.com/gangdexyuhi/client-server-practice
Папка работы: practice_07_client_server/

Цель работы

Собрать полноценное клиент-серверное веб-приложение «Список задач» с полным набором CRUD-операций через браузер, применив все приёмы предыдущих работ: параметризованные запросы, commit/rollback, finally: close(), паттерн PRG и серверную валидацию.

Используемые технологии

- Python 3.13, Flask 3.1: маршруты с параметрами <int:task_id>, methods, redirect, url_for
- SQLite, sqlite3.Row, SQL-выражение CASE WHEN для переключения статуса
- Jinja2, HTML-формы, CSS, JavaScript confirm() для подтверждения удаления
- Протокол HTTP: GET, POST, коды 200, 302, 404, 405

Структура проекта

```
src/
├── init_db.py           # создание базы
├── check_db.py          # проверка структуры
├── crud_console.py      # консольный доступ к той же базе
├── app.py               # Flask-приложение (полный CRUD)
├── app.db
├── requirements.txt
└── templates/
    └── index.html       # форма, список задач, кнопки действий
```

Описание реализации

Код разделён на две части. Слой работы с базой состоит из функций fetch_all_tasks(), insert_task(), toggle_task_status() и delete_task_by_id(); каждая сама открывает соединение, выполняет запрос в try, вызывает rollback() в except и закрывает соединение в finally. Маршруты Flask только

получают запрос, вызывают нужную функцию и возвращают шаблон или редирект.

Таблица 1 — Маршруты приложения

Маршрут	**Методы**	**Функция**	**Действие**
/	GET, POST	index()	GET — список задач; POST — добавление (INSERT) с проверкой названия
/toggle/<int:task_id>	POST	toggle_done()	UPDATE с CASE WHEN: new ↔ done
/delete/<int:task_id>	POST	delete_task()	DELETE задачи, подтверждение в браузере

Переключение статуса выполняется одним запросом UPDATE tasks SET status = CASE WHEN status = 'done' THEN 'new' ELSE 'done' END WHERE id = ?. Функции изменения возвращают rowcount: если он равен 0, маршрут показывает сообщение «Задача #N не найдена». Действия, меняющие данные, оформлены как POST-формы, а не ссылки, поэтому открыть /delete/1 в адресной строке нельзя — сервер вернёт 405.

```
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        title = request.form.get('title', '').strip()
        if not title:
            return render_template('index.html', items=fetch_all_tasks(),
                                   error='Название задачи не может быть
пустым.')

        success, message = insert_task(title)
        if not success:
            return render_template('index.html', items=fetch_all_tasks(),
error=message)

        return redirect(url_for('index'))

    return render_template('index.html', items=fetch_all_tasks())


@app.route('/toggle/<int:task_id>', methods=['POST'])
def toggle_done(task_id):
    affected = toggle_task_status(task_id)
    if affected == 0:
        return render_template('index.html', items=fetch_all_tasks(),
                               error=f'Задача #{task_id} не найдена.')
    return redirect(url_for('index'))


@app.route('/delete/<int:task_id>', methods=['POST'])
def delete_task(task_id):
    affected = delete_task_by_id(task_id)
    if affected == 0:
        return render_template('index.html', items=fetch_all_tasks(),
```

```
                                    error=f'Задача #{task_id} не найдена.')
    return redirect(url_for('index'))
```

Ход работы и результаты запуска

1. Добавил через форму четыре задачи (операция Create).

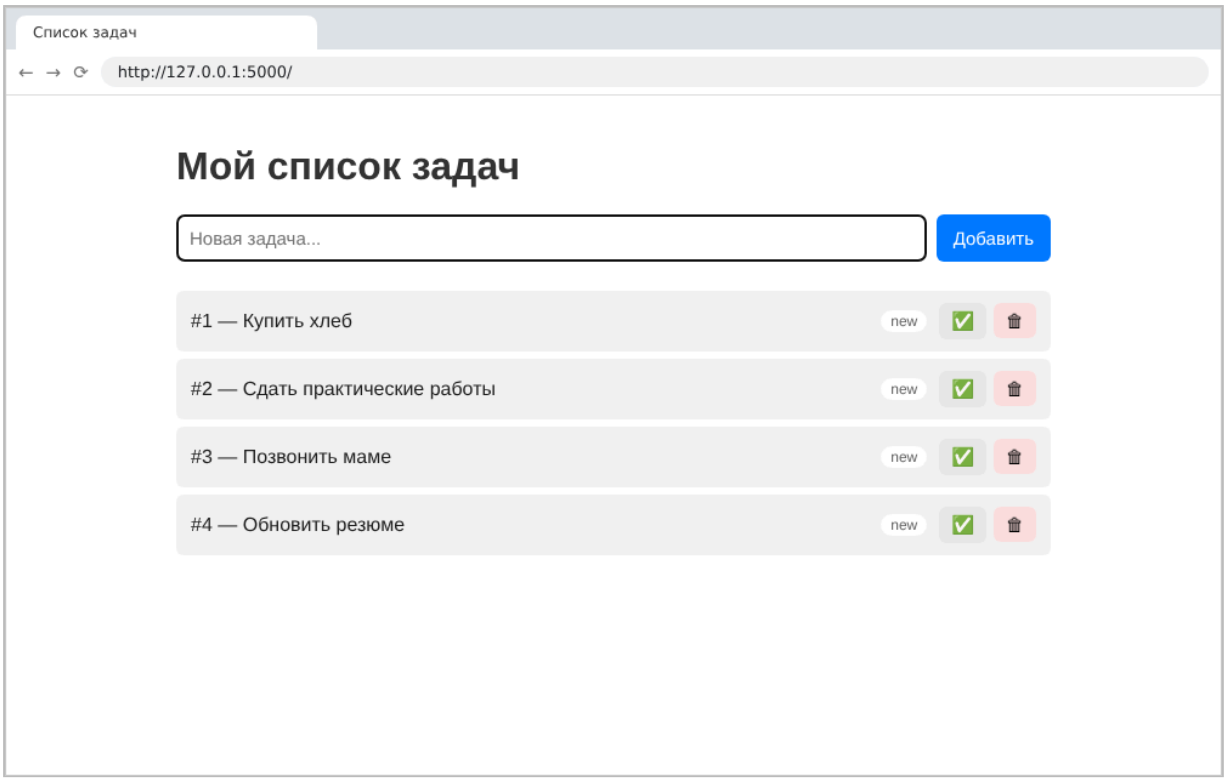


Рисунок 1 — Создание задач

2. Нажал кнопку ✅ у двух задач — статус сменился на done, текст зачёркнут, иконка поменялась на ↩ (операция Update).

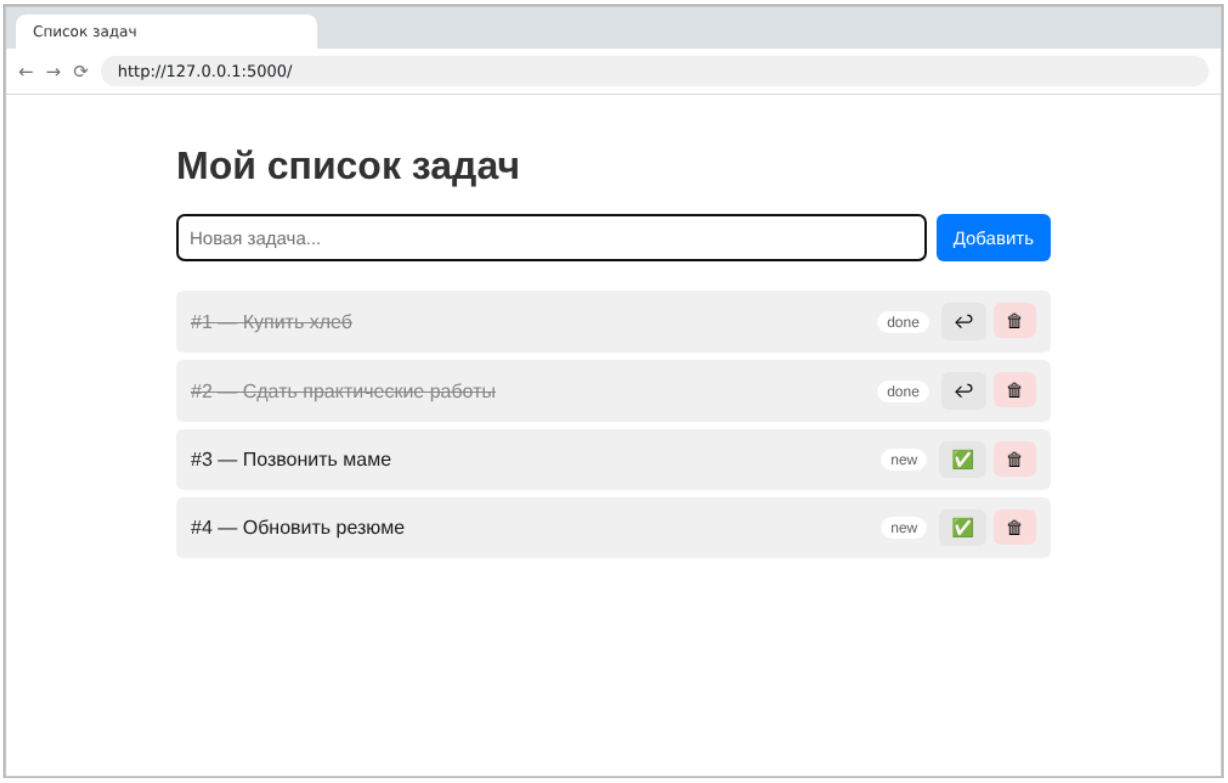


Рисунок 2 — Отметка задач выполненными

3. Нажал ↩ у второй задачи — статус вернулся в new.

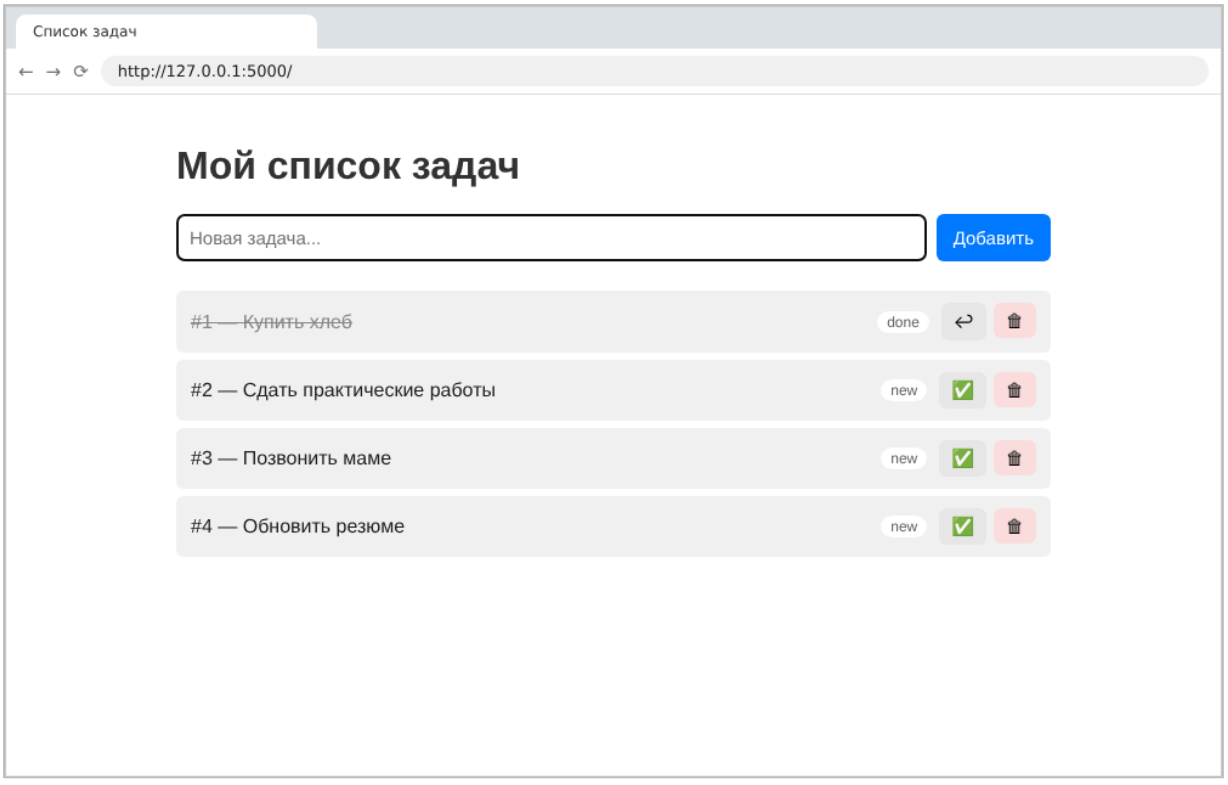


Рисунок 3 — Возврат статуса new

4. Удалил задачу кнопкой 🗑 : браузер запросил подтверждение «Удалить задачу #3?», после согласия задача исчезла (операция Delete).

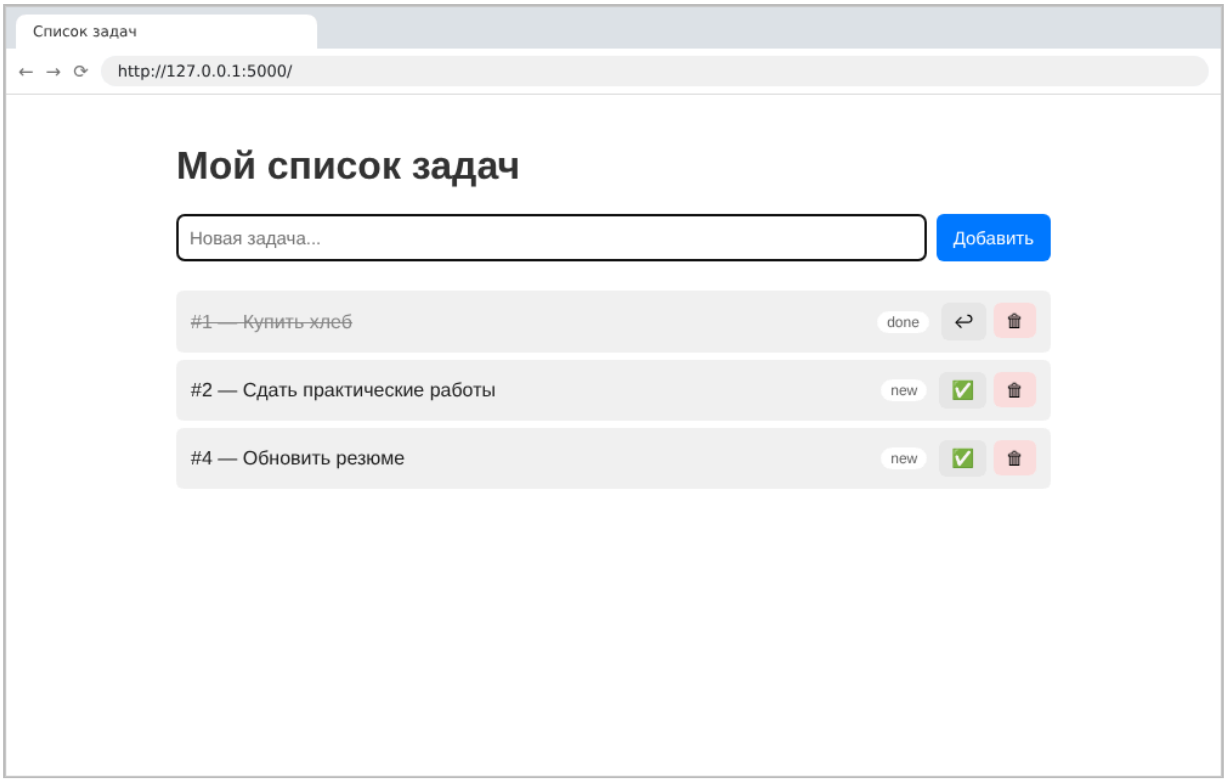


Рисунок 4 — Список после удаления задачи

5. Проверил защиту от неверных запросов: GET-запрос /delete/1 из адресной строки вернул 405 Method Not Allowed, а /toggle/abc — 404 Not Found, так как конвертер <int:...> не принимает нечисловое значение.

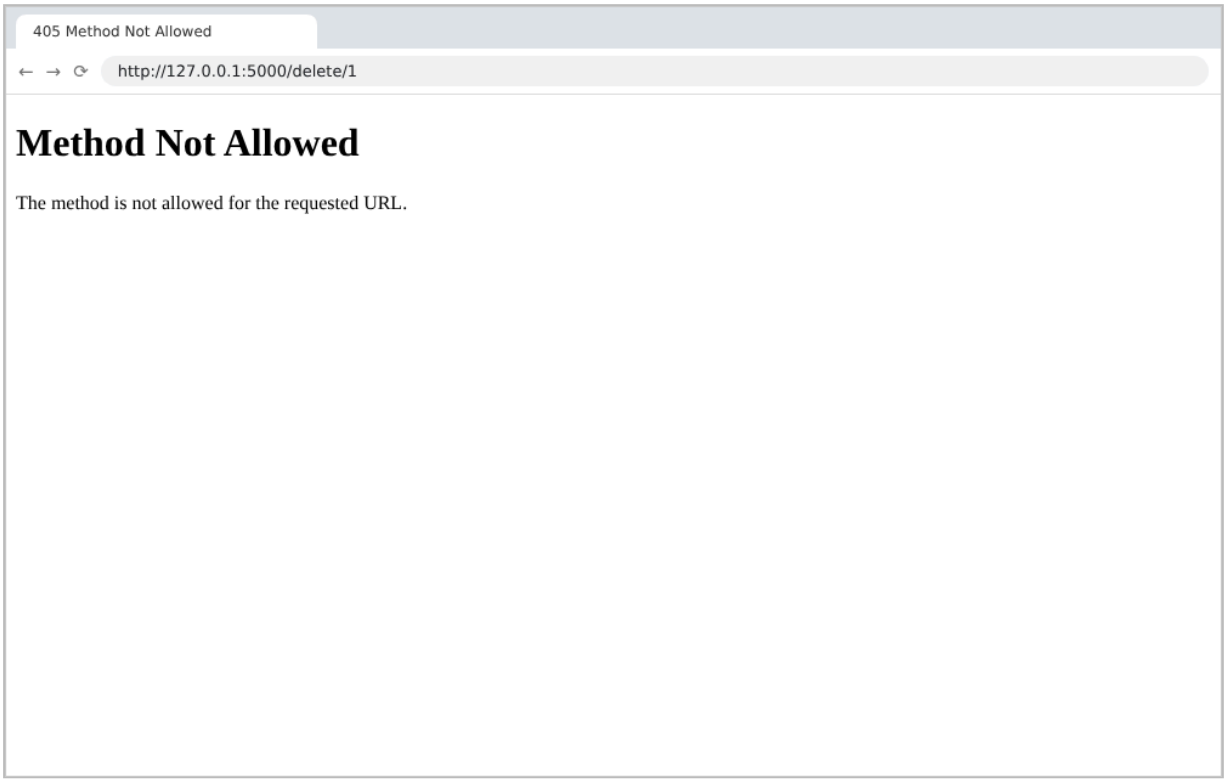


Рисунок 5 — Ответ 405 на GET-запрос к /delete/1

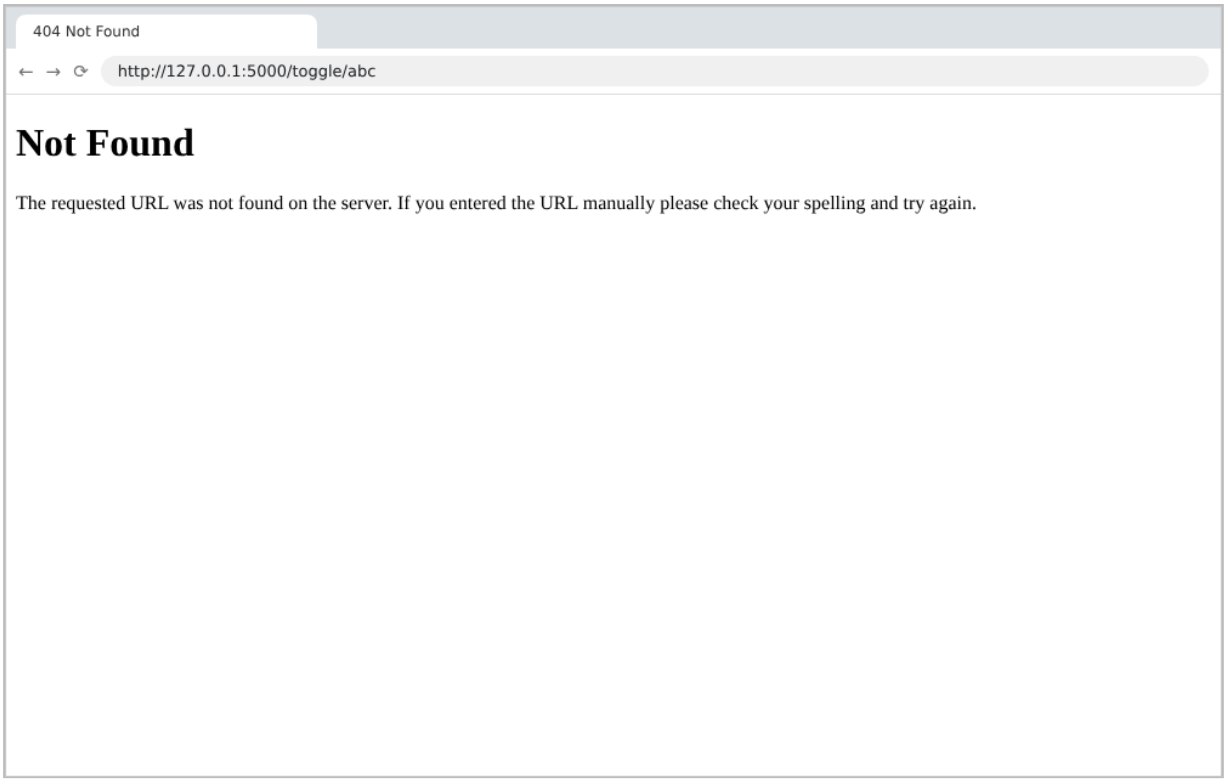


Рисунок 6 — Ответ 404 на /toggle/abc

6. Проверил обработку несуществующего id: во втором терминале удалил задачу #4 через crud_console.py, затем на необновлённой странице нажал у неё кнопку ✅. UPDATE вернул rowcount = 0, и приложение показало сообщение «Задача #4 не найдена».

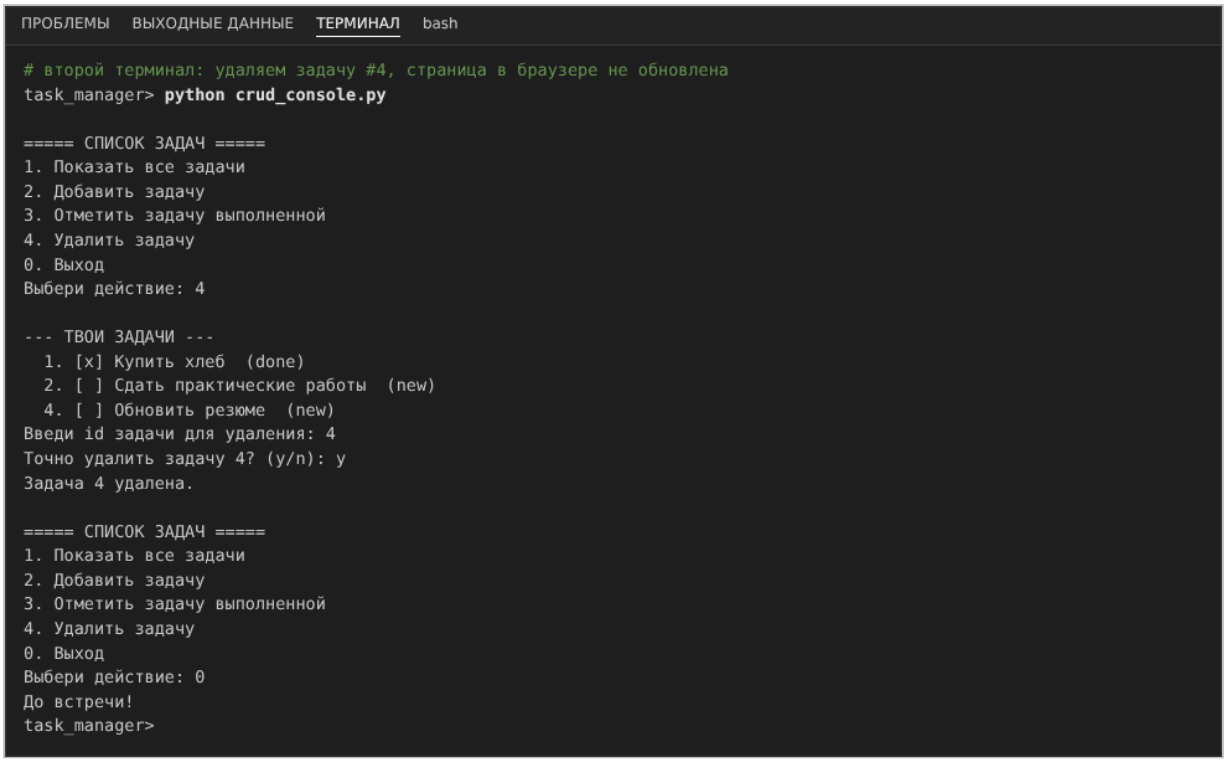


Рисунок 7 — Удаление задачи из консоли

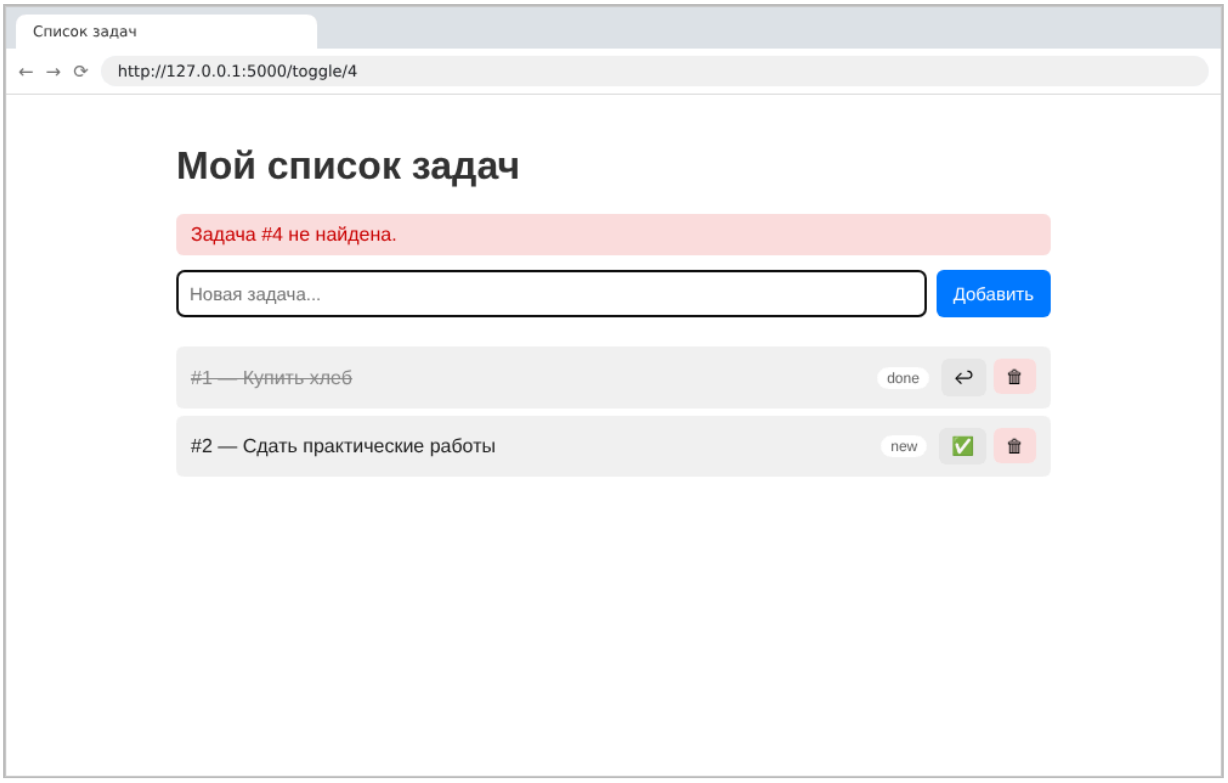


Рисунок 8 — Сообщение о несуществующей задаче

7. Добавил задачу с названием '); DROP TABLE tasks; -- — она сохранилась как текст.

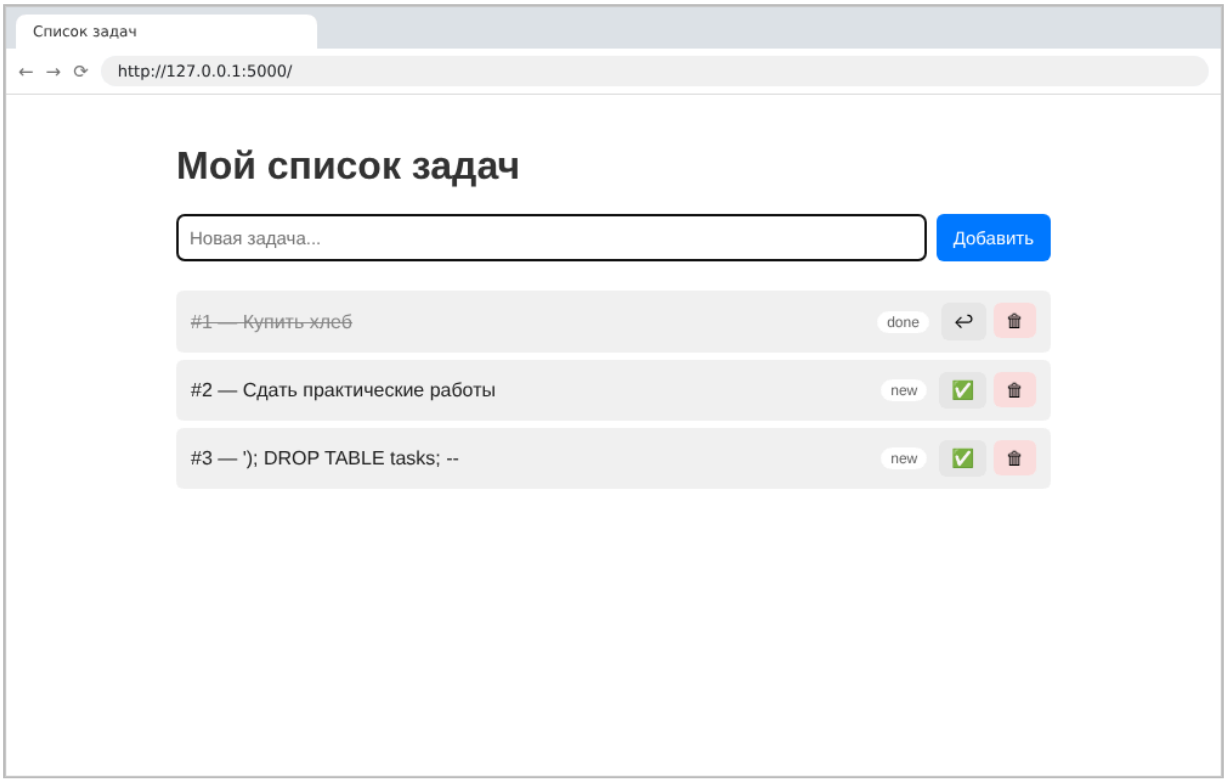


Рисунок 9 — SQL-инъекция сохранена как текст

8. Проверил «живучесть»: пять раз отправил форму с пустым названием, обойдя required. Каждый раз сервер возвращал сообщение об ошибке, после чего crud_console.py без проблем добавил новую задачу — база не заблокирована.

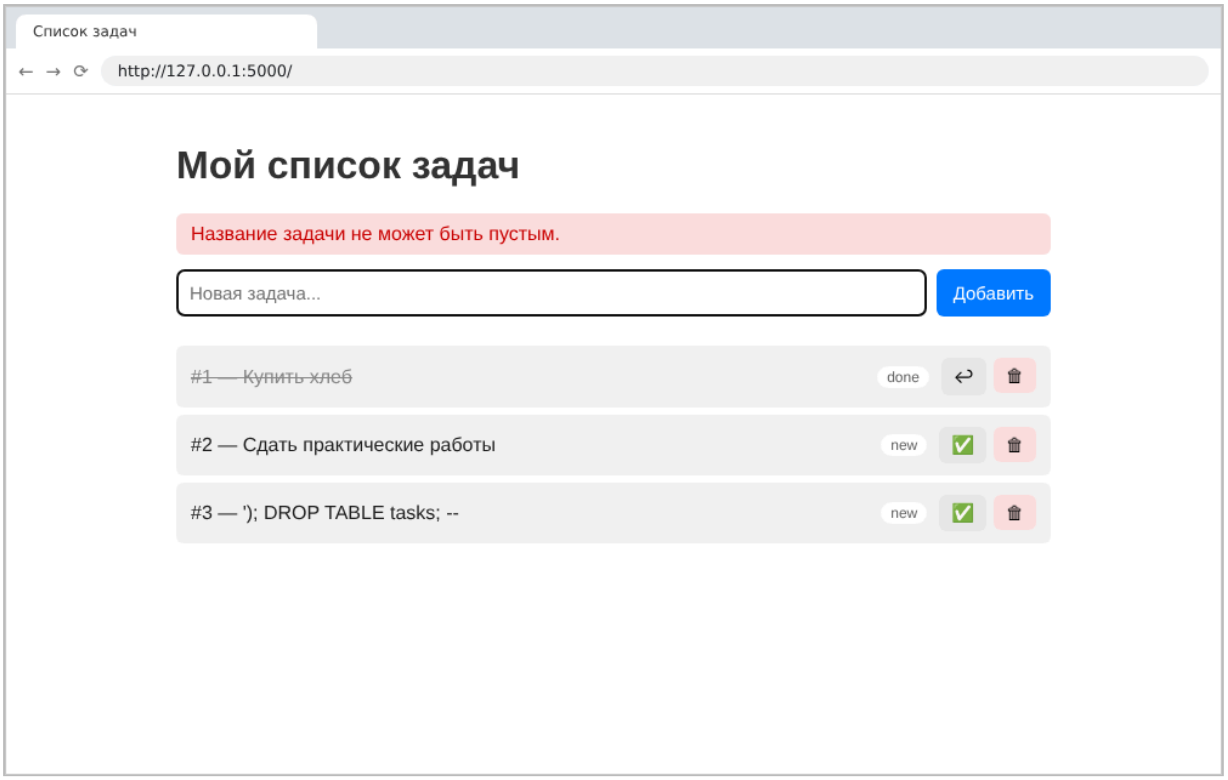


Рисунок 10 — Сообщение о пустом названии

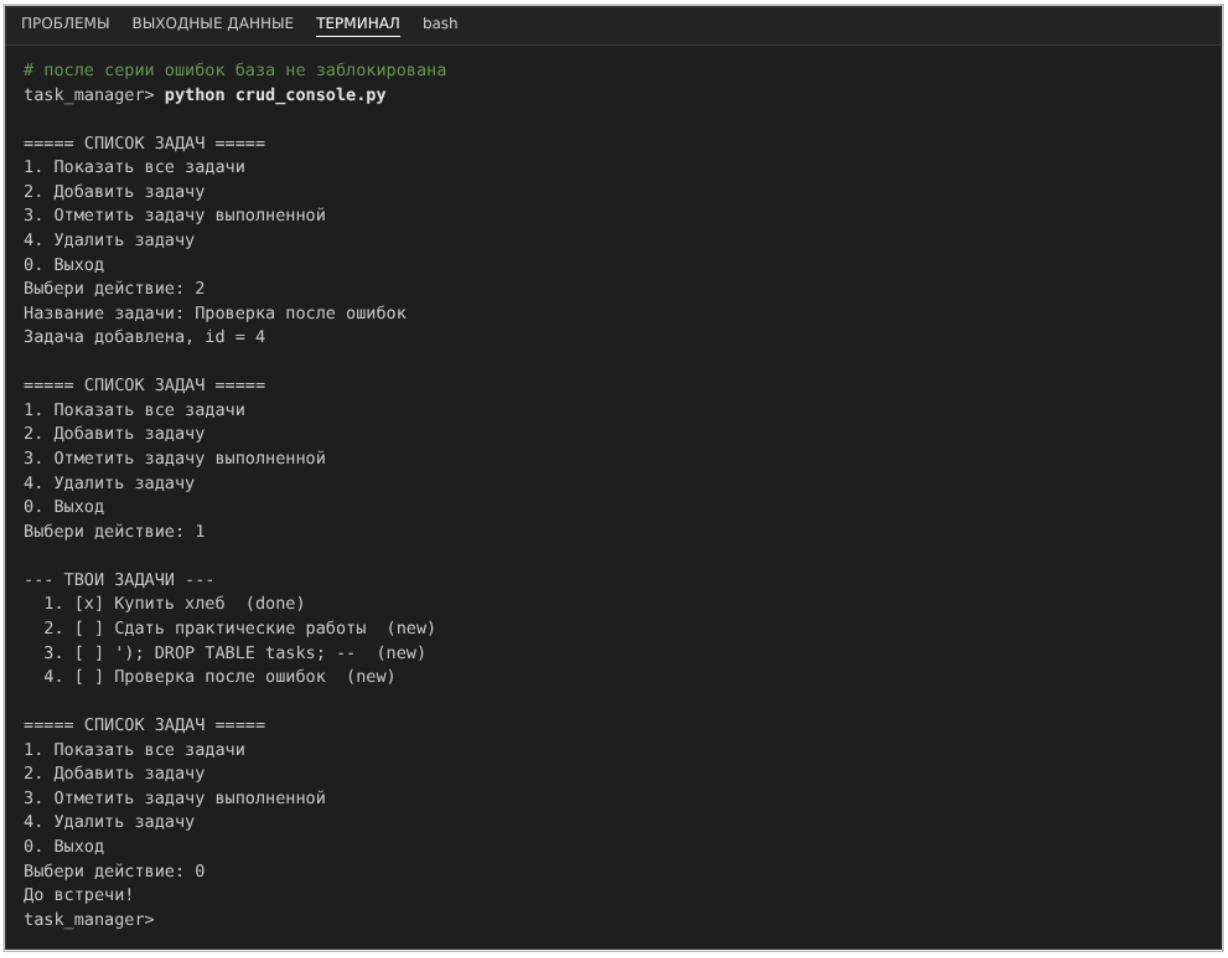


Рисунок 11 — Работа с базой после серии ошибок

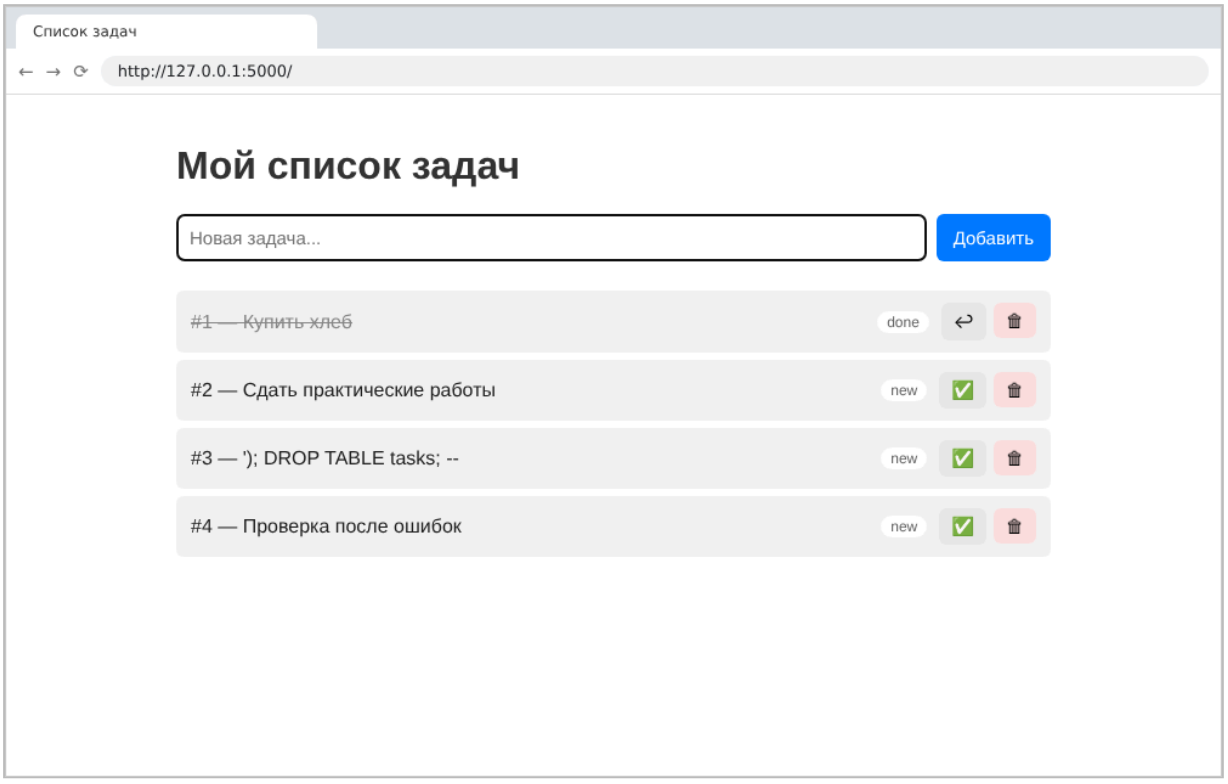


Рисунок 12 — Итоговое состояние списка задач

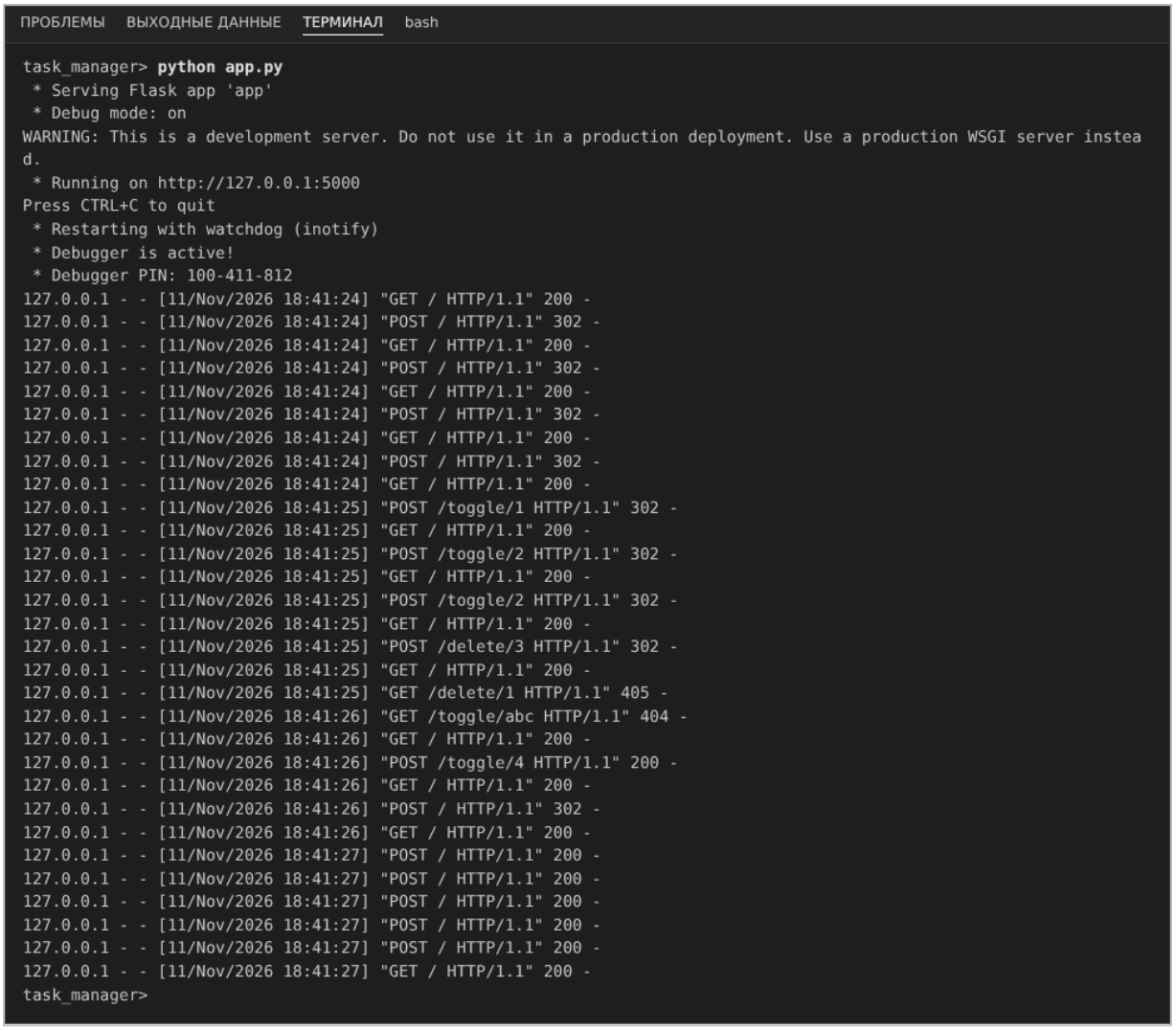


Рисунок 13 — Журнал сервера: все запросы и коды ответов

Примеры запросов и ответов

Таблица 2 — Примеры HTTP-запросов и ответов сервера

Запрос	**Код ответа**	**Заголовок Location**	**Результат**
POST / form: title='Новая задача'	302 FOUND	/	Задача создана, редирект на список
POST /toggle/1	302 FOUND	/	Статус переключён, редирект на список
POST /toggle/9999	200 OK	—	Страница с сообщением «Задача #9999 не найдена»
GET /delete/1	405 METHOD NOT ALLOWED	—	Удаление по GET запрещено
GET /toggle/abc	404 NOT FOUND	—	abc не является числом
POST /delete/1	302 FOUND	/	Задача удалена, редирект на список

Таблица 3 — Проверка полноты CRUD

Операция	**Действие в браузере**	**Результат**
C (Create)	Ввод названия + «Добавить»	Новая строка в списке и в app.db
R (Read)	Открытие /	Список совпадает с содержимым app.db
U (Update)	Кнопка ✅ / ↩	Статус изменился, rowcount = 1
D (Delete)	Кнопка 🗑 + подтверждение	Строка удалена, rowcount = 1

Ответы на контрольные вопросы

1. Почему действия «переключить статус» и «удалить» отправляются через POST-форму, а не через <a href>?

Ссылка выполняет GET-запрос, а GET по правилам HTTP не должен менять данные. Ссылку может случайно открыть пользователь, поисковый робот или браузер при предзагрузке, и данные изменятся без ведома пользователя. Поэтому изменяющие действия выполняются только POST-запросом.

2. Что делает конвертер <int:task_id> в маршруте и почему с ним не нужен isdigit()?

Конвертер принимает только целое число из части URL, сам преобразует его в int и передаёт в функцию аргументом task_id. Если в URL не число, маршрут не совпадает, и Flask возвращает 404, поэтому ручная проверка не нужна.

3. Что означает rowcount = 0 после UPDATE или DELETE?

Запрос выполнился без ошибок, но не нашёл ни одной строки с указанным id, то есть задачи не существует. Приложение в этом случае показывает сообщение «Задача не найдена».

4. Почему каждая функция работы с базой открывает своё соединение, а не получает его параметром?

Так каждая функция самостоятельна: сама управляет транзакцией и гарантированно закрывает соединение в finally. Соединения не живут дольше одного запроса, не путаются между запросами и не остаются открытыми при ошибке. Код маршрутов при этом проще.

5. Где в toggle_task_status находится commit(), и почему без него статус не сохранится после перезапуска?

commit() вызывается внутри блока try сразу после execute с UPDATE. Без него изменение останется в незавершённой транзакции и будет отменено при закрытии соединения, поэтому после перезапуска статус окажется прежним.

6. Что произойдёт, если открыть /delete/3 в адресной строке браузера (GET)?

Маршрут принимает только POST, поэтому сервер вернёт 405 Method Not Allowed, и задача не будет удалена.

7. Зачем в шаблоне onsubmit="return confirm(...)" и можно ли обойти эту защиту?

Это подтверждение защищает от случайного нажатия: если пользователь нажмёт «Отмена», форма не отправится. Это защита только на стороне браузера: её можно обойти, отключив JavaScript или отправив POST-запрос напрямую. От злонамеренных действий она не защищает — для этого нужна авторизация на сервере.

8. Почему return redirect(url_for('index')) стоит после try/finally, а не внутри try?

Редирект нужно выполнить только при успешном завершении операции. Если ошибка произошла, функция возвращает страницу с сообщением из блока except. В текущей версии операции вынесены в отдельные функции, и маршрут делает редирект, только если функция сообщила об успехе.

Вывод

В итоговой работе я собрал клиент-серверное веб-приложение «Список задач» с полным CRUD через браузер. В приложении используются параметризованные запросы, транзакции с commit и rollback, гарантированное закрытие соединений, паттерн PRG, серверная валидация и маршруты с параметрами. Проверка показала, что приложение корректно обрабатывает неверные запросы (404, 405), несуществующие id и SQL-инъекции, а после серии ошибок база не блокируется.